

on has been completed

Weights

We use a lightweight system of issue weighting with the knowledge that things take longer than you think.

It's OK if an issue takes longer than the weight indicates. The weights are intended to be used in aggregate, and what takes one person a day might take another person a week, depending on their level of background knowledge about the issue. That's explicitly OK and expected.

These weights we use are:

Weight	Meaning
---	---
1	Trivial, does not need any testing
2	Small, needs some testing but nothing involved
3	Medium, will take some time and collaboration
4	Substantial, will take significant time and collaboration to finish
5	Large, will take a major portion of the milestone to finish

Anything larger than 5 should be broken down if possible.

Priorities

We use priority labels to provide the team and other contributors with an approximate timeline for when we intend to address issues.

Label	Meaning
---	---
~"ProdSecEngPriority::1"	This issue requires immediate attention and should be worked on as soon as possible. Typically, it is added to the next milestone or addressed during the current one if a team member has extra capacity.
~"ProdSecEngPriority::2"	This issue is essential but can wait for a few milestones.
~"ProdSecEngPriority::3"	This issue is a nice-to-have, but we do not commit to a specific timeline.

Choosing what to work on

Product Security Engineering should always have at least one AppSec-related issue in flight. This rule's intention is to make sure we achieve our mission of reducing AppSec's manual work burden.

If a Product Security Engineer completes all their work in a Milestone, they should take an item from the top of the backlog and assign themselves to it. If they need to stop working on something they should unassign themselves, @ mention the team, and apply the correct workflow label (e.g. ~workflow::blocked).

Merge Request Reviews

When contributing to a project that is owned or maintained by another team or an official

GitLab asset, we follow that project's established review conventions, rules, and requirements.

When contributing to a project owned and primarily maintained by Product Security Engineering:

- We default to asking for other Product Security Engineering team members to review our merge requests
 - We strive to review each other's contributions in order to encourage collaboration, facilitate knowledge sharing, and reduce silos
 - We must acknowledge that we are a small team and that sometimes a thorough review isn't going to happen in a timely manner, impacting velocity
 - We evaluate the tradeoff between knowledge sharing and velocity on a case-by-case basis, with each team member empowered to make decisions on foregoing a review
 - We can skip a formal review if something is blocking, time-sensitive, and/or resolving an urgent high-impact need
 - We try to pick up issues in tooling other team members have written

Project namespaces

The Product Security Engineering team defaults to using the namespaces of the stakeholder that we're performing the work for:

- For contributions to GitLab assets, we contribute to the relevant repositories where other work is performed
- For AppSec related work, we use the AppSec tooling namespace unless there is a compelling reason not to
- For our team's repositories, we use the Product Security Engineering tooling namespace

Tooling Integration work

One of the key focuses of our team is integrating custom out-of-product tooling that Product Security teams use into GitLab.

Identification and Planning

This epic is used to organize, plan, and track the work needed to implement custom Product Security tooling into GitLab.

For each custom tool:

- A tooling integration epic is created and made a child of the Product Security Tooling Integration Epic
- An entry is also added to the appropriate table in the List of Tools for the Product Security Tooling Integration Epic, if it does not already exist
- An epic is created for each piece of discrete functionality of that tool and added as a child to that tool's integration epic
 - In some cases, we may need separate epics for specific components, modules, or other foundational work that doesn't fit specifically into a particular bit of functionality
 - Any other work required to implement, roll out, or handover that functionality should be tracked as children of these epics
 - Each of these epics must be labeled with ~ProdSecEngMetric::Tooling Integration

- A tooling handover epic must be created so that conversations with development teams can begin as soon as possible
- A sunset roadmap is created, along with sunseting issues

Discrete Pieces of Functionality

As best we can, we try to break these tools down into discrete pieces of functionality. The purpose of doing so is to:

- Ensure understanding of the actual functionality and value proposition the tool is bringing
- Break the work down into smaller, actionable chunks of work
- Enable parallelization of work, cross-team collaboration, and decrease organizational friction

We define discrete pieces of functionality as a piece of functionality that is a distinct, separate thing that the application does. This functionality may be user facing or a necessary part that enables the rest of the application -- although it is not a component. One or more components in the codebase may work together to deliver a discrete piece of functionality.

Taking a basic todo list as an example, we might break it down into at least a few pieces of discrete functionality:

- Creation of todo list items
- Viewing of todo list items
- Editing of todo list items
- Removal of todo list items
- Rearranging of todo list items
- Tagging or categorization of todo list items

We do this on a best-effort basis, and acknowledge that not every tool may fit nicely into separate buckets of functionality. Product Security Engineering team members should feel empowered to be flexible in how we plan, chunk, and deliver work in an efficient and effective manner.

Tooling Handover Epics

For any functionality being built into the product, we need to communicate and collaborate with the development teams that will own and maintain it going forward. In order to accomplish this, we create tooling handover epics and add them as children to the appropriate tooling integration epics.

These tooling handover epics will:

- Be created as early in the process as possible
- Establish a single source of truth for all handover coordination and decisions
- Help us identify what teams would own each specific piece of functionality or component
 - The appropriate product and engineering managers for the teams should be directly pinged
- Enable us to collaborate on a definition of done, feature flags, rollout, and eventual handover

Sunsetting Issues

In order to keep track of the high-level requirements to fully deprecate a custom tool, we create issues specifically to track the sunseting of that tool. This is done using the sunseting template in the Product Security Engineering team repository.

Once a particular sunseting requirement has been fulfilled, the Product Security Engineering team needs to close the relevant sunseting issue in the Product Security Engineering team repository.

Step-by-step processs

When beginning work to implement custom tooling into the product, we take the following actions:

- Create a tooling integration epic and make it a child of the Product Security Tooling Integration Epic
- Create a tooling handover epic and make it a child of the newly created tooling integration epic

As we begin to build context on the tool, as part of our planning phase we will:

- Create and document an appropriately sized Architecture Design Workflow for the tool, if one does not yet exist
- Identify and create an epic for each piece of discrete functionality or required component
 - Make these epics children of the tooling integration epic
 - Be sure each of these have the ~ProdSecEngMetric::Tooling Integration label
- Make a quick, best-guess recommendation as to what pieces of functionality or components we should start working on first
- Create a sunset roadmap and sunseting issues

Once we have identified the functionality and components for this tool, we:

- Create issues under each functionality or component epic that push us towards integrating that functionality
- Reach out to Product and Engineering managers in the tooling handover epic to give them visibility, discuss any overlap with in-flight or planned work, and discuss eventual handover details
 - Does the functionality described in the epics/issues align with work their group does? If not, do they have suggestions as to who we should follow up with?
 - Do they have any concerns about the functionality, from a product/engineering/maintenance perspective?
 - What is the agreed upon definition of done, and at what point would it make sense to hand this work over to their team?
 - What would they like to see from us in order to ensure a smooth handover?

After we have finished a contribution related to a custom tooling integration, we need to:

- Update the sunseting roadmap to indicate that a particular task or phase has been completed
- If appropriate, close the relevant sunseting issue

Milestone Planning

Please visit the Product Security Engineering Milestone Planning page for information about how we plan our work.

Metrics

Please refer to the Product Security Engineering Metrics page for information on our metrics.

Integrating Product Security Team Requirements Into GitLab

Information on how the Product Security Engineering team plans, develops, and maintains functionality required by the GitLab Product Security sub-department can be found here.

Open Source Contributions

When possible, we contribute new features or security improvements directly to the dependencies that GitLab relies on so that everyone can benefit from those enhancements. Since those contributions happen in external repositories, they can't be tracked with our labels. In those cases, we should apply the appropriate labels to the merge requests we create for updating the dependency version to the one that includes our contributed changes.

SECTION: security/product-security/security-platforms-architecture/product-security-engineering/metrics.md

Metrics

The Product Security Engineering team currently tracks the following metrics:

- Number of Merge Requests merged into GitLab products directly related to implementing the functionality required by Product Security teams
- Number of Merge Requests for creating security enhancements, defense in depth measures, and paved roads
- The percentage of distinct value propositions in current in-house custom tools that have been contributed to the product

These can be observed in this Tableau dashboard.

Metrics Labels

Labels are a driving force behind our metrics collection. The appropriate label must be applied to the relevant issue, merge request, or epic.

| Type of Work | Label | Where to apply label |

| --- | --- | --- |

| Product Security team requirement: functionality within the product required by GitLab

Product Security teams | ~ProdSecEngMetric::ProdSecRequirement | Issues and Merge Requests, sometimes Epics |

| Defense in Depth: an MR that modifies existing non-vulnerable functionality to be more robust if an "earlier" security control fails | ~ProdSecEngMetric::Defense in Depth | Issues and Merge Requests, sometimes Epics |

| Paved Road: a new tool, method, class, check that, gives GitLab's contributors an easier way to perform an activity securely | ~ProdSecEngMetric::Paved Road | Issues and Merge Requests, sometimes Epics |

| Tooling Integration: work done as part of the effort to integrate functionality from custom in-house tooling into a GitLab product | ~ProdSecEngMetric::Tooling Integration | Issues and Merge Requests, sometimes Epics |

| Custom Tooling: work performed to build, maintain, or augment outside-of-the-product custom tooling needed to satisfy Product Security requirements | ~ProdSecEngMetric::Custom Tooling | Issues and Merge Requests, sometimes Epics |

| Pending: the work type isn't entirely clear yet, but we don't want to block progress for now | ~ProdSecEngMetric::Pending | Issues, Merge Requests, and Epics |

| Sunsetting: for issues that represent a specific feature or piece of functionality required to eventually deprecate a custom tool used by Product Security teams | ~ProdSecEngMetric::Sunsetting | Issues in the product-security-engineering-team repo |

| Internal: for team tasks, e.g. processes & planning | ~ProdSecEngMetric::Internal | Issues and Merge Requests, Epics |

SECTION: security/product-security/security-platforms-architecture/product-security-engineering/milestone-planning.md

Milestone Planning

The Product Security Engineering team plans its work on a cadence based around GitLab Product Milestones. This page describes the planning process that we use to determine what work will be completed for each Milestone.

Milestone Planning Issue

For each Milestone, a Milestone Planning issue is created in the Product Security Engineering team repository. The purpose of this issue is to:

- Identify potential work to perform
- Identify refinement gaps and determine how to address them
- Determine what work we're committing to for the upcoming Milestone
- Set and communicate priority for the work we've decided to take on

This issue is the single source of truth for all planning related discussions and decisions related to the upcoming Milestone.

Milestone Planning Process

1. An issue will be created using the Milestone Planning issue template
 1. The issue will be created immediately after planning is finished for the current Milestone
 1. Throughout the current Milestone, Product Security Engineering team members can add work

to the upcoming Milestone's Parking Lot

1. The Product Security Engineering manager will be responsible for completing the checklist items in the Planning Checklist section of the Milestone Planning issue

1. Product Security Engineering team members will add any work being carried over from the previous Milestone into the Milestone Work table

1. The Product Security Engineering team will add potential work items to the Parking Lot section, with a brief explanation of why it would be good to include in the Milestone

1. Team members can add discussion threads to discuss potential work to pull into the Milestone

1. Both individual team members and the Product Security Engineering manager can add items to this list

1. The Product Security Engineering team will work together to add new items to the Milestone Work table

1. Each item being added must be refined before it can be formally committed to

1. The team member likely to take on the work should review and agree with the Weight, if it wasn't them who refined the issue.

1. Once we have refined and committed to the work, the relevant issue needs to be updated with the Milestone, Assignee(s), and Metrics label

1. The Milestone Planning issue should be finalized at least 3 days before the Milestone Start Date

1. The Product Security Manager will use threads in the Milestone Planning issue to work with each Product Security Engineering team member to finalize their workload

1. Once finalized, the Planning Issue should be closed and an issue for the next Milestone should be opened

Milestone Planning Responsibilities

Product Security Engineering team members are responsible for:

- Evaluating and communicating their capacity for the Milestone (based on PTO and other factors)
- Adding work that is being carried over into the Milestone Work table
- Adding potential work items to the Parking Lot and being involved in discussions around what work we should pull into the Milestone
- Collaborating with the Product Security Engineering manager to finalize the set of work being committed to for the Milestone

The Product Security Engineering manager is responsible for:

- Creating, updating, and maintaining the Milestone Planning issue
- Collaborating with Product Security Engineering team members to discuss potential work, identify refinement gaps, and assemble the Milestone Work table
- Coordinating the finalization of the Milestone Planning issue

Retrospectives

After each Milestone, the Product Security Engineering team participates in an asynchronous retrospective. The purpose of this retrospective is to recognize things that went well, recognize things that didn't go well, and to identify changes that we should implement.

Retrospectives are captured in sub-tasks to the relevant Milestone Planning issues. Team members may choose to add items to the retrospective over the course of the Milestone, but should also feel empowered to concentrate on the Milestone work itself and think about retro items later.

By default, these retrospectives should have a due date of 2 weeks after the Milestone end date. However, team members can always add items to the retrospective after the due date.

SECTION: security/product-security/security-platforms-architecture/product-security-engineering/product-security-requirements.md

Integrating Product Security Team Requirements

Product Security teams at GitLab have responsibilities and processes that require engineering effort to perform or operationalize. The Product Security Engineering team is responsible for building solutions that suit Product Security team needs.

Ideally these solutions will be contributed in the form of material improvements to GitLab, since we aim to drive down GitLab's software security risks by securing GitLab with GitLab. In doing so, we enable Product Security teams to dogfood GitLab functionality, identify improvements that additionally help our customers, and reduce the need for tooling developed outside of the product.

Goals

- Satisfy the needs of GitLab Product Security teams
- Improve the GitLab product, adding functionality and features that directly impact product security teams
- Make impactful contributions to the GitLab product quickly and iteratively, minimizing organizational friction and overhead
- Increase Product Security sub-department use of GitLab features in day-to-day operations and processes

Pilot and Integration Engineering

The processes described on this page are the building blocks for the Product Security Engineering team to move towards distinguishing two core engineering focus areas: Pilot and Integration Engineering.

Pilot Engineering

Pilot Engineering is defined as the work required to deliver the desired outcome as quickly as possible, with modest regard to typical GitLab Engineering standards. This does not mean low-quality engineering: documentation, automated testing, minimizing tech debt, and other best practices are still expected with this work.

The following are some examples where Pilot Engineering work would tend to be less-than-ready for contribution:

- Design, user experience, and interface
- Comprehensive production readiness
- API, identity, and role based access control compatibility
- Product vision and roadmap alignment

In building in-house tools that exist outside of the GitLab product, some Product Security teams have already delivered work that can be considered Pilot Engineering. These tools are actively used by the Security division to perform certain responsibilities and deliver on commitments. The incorporation of those existing tools into the GitLab product is considered Integration Engineering, and is being tracked in this epic following the process described here.

Integration Engineering

Integration Engineering is the effort required to take Pilot Engineering work and contribute the features and functionality to GitLab itself.

The expectation is that Product Security Engineering will be responsible for maintenance and extension of these functionalities and features until a formalized handover to a GitLab Product team occurs, as indicated in the processes described on this page.

Process

When a Product Security team identifies a need that is not satisfied by the current GitLab product or an existing tool, the process below will be followed.

Planning

- Requesting team submits an issue using this template in the Product Security Engineering team repository
- The Product Security Engineering team follows their defined refinement workflow, while collaborating with the requesting team to understand the requirements
- A decision is made as to whether Product Security Engineering should build the solution, if we should wait for a GitLab development team to implement it, or if it should be purchased externally
 - If the decision involves Product Security Engineering action, it will be scheduled into an upcoming Milestone based on priority, urgency, and capacity
- A decision is made as to whether or not the feature or functionality is to be put behind a GitLab Product Security specific feature flag, indicating it is initially meant to be used exclusively by the GitLab Product Security team
 - The intent of such a feature flag is to ensure that product fit and customer usability does not prevent the implementation of functionality required by GitLab Product Security teams
 - This will require discussion among all stakeholders, and be documented in the requirement request issue
 - This will require that a handover and transition plan is agreed upon by the Product Security Engineering team and the relevant GitLab Product and Engineering managers

Development and Release

- The Product Security Engineering team will implement the requirements as part of their

regular workflow, observing established GitLab contribution standards

- If the feature or functionality is determined to be exclusively for use of the GitLab Product Security team, it will be put behind a feature flag explicitly for this purpose
 - The standard feature flag process and requirements will be followed
- The Product Security Engineering team will own, maintain, and support the feature or functionality depending on the need for a Product Security specific feature flag or other decisions made during planning

Ownership, Maintenance, and Transition

This section describes the ownership, maintenance, and transition of product contributions made to satisfy GitLab Product Security team requirements.

- To the extent that implemented capabilities are contained exclusively within GitLab Product Security owned namespaces and/or enabled exclusively for GitLab Product Security teams, the Product Security department will be responsible for maintenance and upkeep
 - The team primarily responsible for this will be the Product Security Engineering team, supported by the requesting team where possible
 - This applies to features and functionality that are for the exclusive use of the Product Security teams, which may be appropriate if:
 - The requirement is very specific to the GitLab Product Security team and would have little external value, with one indicator being a very low Reach score
 - The requirement is inconsistent or incongruent with current Product vision
 - The requirement needs to be incubated by the GitLab Product Security team for other reasons
 - The Planning process will require that stakeholders from the requesting Product Security team, the Product Security Engineering team, and the relevant Product and Engineering teams discuss and collectively make a decision with regards to the need to put the feature or functionality behind a Product Security specific feature flag
- As features and functionality become enabled for and available to users outside of the GitLab Product Security team, an agreed-upon transition will happen to handover ownership and maintenance
 - Standard GitLab processes and expectations around support for experiment, beta, and generally available features will be followed
 - General availability will constitute a complete transfer of ownership responsibilities, although handovers may occur earlier

Build vs. Wait vs. Buy

For a Product Security team requirement, there are three possible actions that can be taken:

Action	Description
Build	The GitLab Product Security Engineering team should plan to build the solution
Wait	Based on a planned or in-flight effort owned by a GitLab development team, we will wait for them to implement a solution
Buy	A third-party vendor tooling or solution may be purchased to satisfy the requirement

When evaluating a potential action, keep in mind that there may be a hybrid approach that could be an opportunity for cross-team collaboration. In some cases, the Product Security Engineering team may be able to parallelize an in-flight effort.

Evaluating an action

Below is a list of considerations for each potential action, with an inexhaustive list of benefits and drawbacks for each one.

Considerations for Build

Benefits

- We have more control over the solution, can focus on building a minimal valuable change that directly satisfies the requirement
- We can start building immediately (depending on urgency, capacity, and priority)
- Product and Engineering teams can focus on other roadmap items to deliver value to customers
- We are contributing to the product, likely solving for a need that some customers have
- Enables dogfooding of Security-persona focused features within the Product Security sub-department

Drawbacks

- There is an opportunity cost (what are we not doing in choosing to work on this?)
- This may be too slow of an option, depending on team capacity and the urgent of the requirement
- For parts of the product that is new to the Product Security Engineering team, there is higher overhead for context building (decreased efficiency, potential for tech debt)
- The solution we arrive at may differ from the Product/Engineering team's vision or ideal solution

Considerations for Wait

Benefits

- The Product Security Engineering team can focus on making other contributions
- Product Security can be an early adopter, provide shorter feedback loops, participate in usability studies, or otherwise offer insights that help development teams deliver better results for customers
- Enables dogfooding of Product features within the Product Security sub-department
- Promotes cross-team collaboration and interaction via discussions and feedback

Drawbacks

- We may have to wait longer for the solution to become available, depending on timelines
- Priority for the Product teams may change, resulting in a slower or indefinite delivery of the solution
- The solution may not be an exact fit, and may still require further iteration or extension from the Product Security Engineering team

Considerations for Buy

Benefits

- The requirement may not fit into the GitLab product, conflict with the product vision, or not be capable within the product (resulting in build being an unviable action, and buying being the only option)
- It may be a more robust, feature-rich solution compared to something that gets built into the product or created as an in-house tool
- Depending on functionality and requirements, procurement may be faster than building internally
- Vendor-provided services and support may be more efficient, enabling teams to focus on other efforts

Drawbacks

- A valid third party solution may or may not exist (or may be excessive in scope)
- Recurring financial commitment to procure and support the tool
- We may find ourselves reliant on the tool and unable to easily change solutions or switch to an eventual in-house offering
- Does not result in product contributions or deliver results for customers via product improvements
- Does not enable Product Security teams to dogfood the product
- Likely reduces the amount of time Product Security teams spend in GitLab products, reducing cross-collaboration and feedback opportunities

Team Responsibilities

The table below describes the responsibilities for each stakeholding team that may be involved in this process.

Team	Responsibility
---	---
Product Security teams	Submitting requirement issues, participating in refinement and solution validation discussions, providing feedback
Product Security Engineering	Refining submitted requirements and potential solutions, facilitating discussions and decision-making with other stakeholders, implementing + maintaining + supporting solutions until handover occurs
GitLab Product and Engineering Managers	Participating in discussions around solutions + ownership + transition, eventually owning functionality and features after handover

Tips and Tricks

Sunset Roadmaps

A sunset roadmap describes the phases and steps needed to move functionality from custom tooling into GitLab. It provides a high-level overview to leadership that answers "what needs to happen so we can secure GitLab with GitLab, instead of with this custom tooling". These steps are optional guidance, not prescriptive.

1. Create a "Sunset Roadmap" issue in the tool's issue tracker when you create the project.
 1. As functionality is added to the tool, update the sunset roadmap. Remember to keep it high level. Technical detail can live elsewhere, like the README.
 1. If the tool has no existing sunset issue, or the existing one is out of date, perform a gap

analysis of functionality between GitLab and the custom tooling

1. Identify the stakeholders that use the tool and involve them early - they can point you to relevant context quickly, and might need to remain informed of sunsetting efforts.

1. The gap analysis might identify requirements that neither the tooling nor GitLab currently meet. These don't need to be added to the Sunset Roadmap.

1. Identify any GitLab issues and/or architectural blueprints that describe plans the product team might have for GitLab functionality

1. Segment the work into "phases".

1. Each phase has the steps needed to move a piece of functionality into GitLab. It will include at least two steps: adding to GitLab, and removing from the custom tooling

1. Link to existing issues, epics, blueprints

1. Each phase should state "This phase is complete when ...".

1. Consider any process changes that might be required.

1. Open sunsetting issues as needed. Each issue is intended to describe a piece of functionality required to deprecate the custom tool, which may align with a particular sunsetting roadmap phase or a culmination of phases.

1. Have the Sunset Roadmap peer reviewed

1. Inform a Product Security Manager that the Roadmap is ready for prioritization. This too can follow the same "Build vs. Wait vs. Buy" process documented above.

Example:

1. <https://gitlab.com/gitlab-com/gl-security/product-security/product-security-engineering/product-security-engineering-team/-/workitems/60/> is an example of a Gap Analysis on two ProdSecEng tools

1. <https://gitlab.com/groups/gitlab-com/gl-security/-/epics/294sunset-roadmap> is an example of a Sunset Roadmap created after the Gap Analysis

SECTION: security/product-security/security-platforms-architecture/product-security-engineering/runbooks/_index.md

This page is a list of runbooks for the Product Security Engineering team.

SECTION: security/product-security/security-platforms-architecture/product-security-engineering/runbooks/hackerone-tooling.md

HackerOne Tooling Runbook

This runbook is for information related to the suite of tools created to help support the Application Security team's HackerOne processes.

Repository List

Information in this runbook may help with debugging, maintaining, or supporting the tools and services found in the following repositories:

- h1bot, which provides import related functionality
 - See also: h1bot configuration and deployment
- h1-gitlab, which performs a number of GitLab issue lifecycle management tasks
- h1-attachments, which serves imported HackerOne attachments
 - See also: h1-attachments configuration and deployment

Re-deploying

h1bot and h1-attachments are deployed using GitLab continuous delivery features. In some cases, re-deploying the environment may fix problems with the tooling, such as request timeouts to the attachments server.

To re-deploy these:

1. Visit the appropriate configuration and deployment project
1. Under the Operate menu option on the left side of the Project view, click on Environments
1. For the live environment, follow the steps to re-deploy to environment

SECTION: security/product-security/security-platforms-architecture/product-security-engineering/runbooks/runway-guidelines.md

Runway tooling guidelines

This runbook is intended to help determine whether a tool is a good fit for Runway. In certain cases, it may be beneficial to use Runway to ease tooling integration work.

What is Runway ?

The proposal of the design document sums up what Runway seems to be today:

> Runway is a means for deploying a service, packaged up as a Docker image to a production environment. It leverages GitLab CI/CD as well as other GitLab product features to do this.

Why Runway?

The executive summary of the Runway blueprint gives a good explanation on the motivation behind it.

> This document proposes a service-integration approach to setting up infrastructure to allow teams within GitLab to build new application features that leverage AI, ML, and data technologies at a rapid pace. [...] The current application architecture runs most GitLab application features in Ruby. However, many ML/AI experiments require different resources and tools, implemented in different languages, with huge libraries that do not always play nicely together, and have different hardware requirements. Adding all these features to the existing infrastructure will increase the size of the GitLab application container rapidly, resulting in slower startup times, increased number of dependencies, security risks, negatively impacting development velocity, and increasing complexity due to different hardware requirements. As an alternative, the proposal suggests adding services to avoid overloading GitLabs main workloads.

These services will run independently with isolated resources and dependencies. By adding services, GitLab can maintain the availability and security of GitLab.com, and enable engineers to rapidly iterate on new ML/AI experiments.

If we extract away the ML/AI parts to any tools that requires different resources and tools than Ruby for its function, it becomes clear that an advantage of Runway is being able to use different technologies than what is typically used within GitLab when appropriate.

The goals section of the design document of Runway gives the following:

- > Development teams aiming to deploy their service without having to worry too much about managing the infrastructure, scaling, monitoring.
- > We are focusing on satellite services that are stateless and thus can be autoscaled to meet demand.
- > We aim to integrate with existing GitLab features and tooling to provide a streamlined experience.

The following list of questions can help determine if a tool may be a potential fit for Runway:

1. Is the tool stateless and does it benefit from being auto scaled?
1. Does it require an infrastructure that does not fit into GitLab?
1. Is there some complexity to managing the infrastructure of the tool that would justify it being outside of GitLab's monolith?
1. Are we adding another layer of complexity by having that infrastructure outside GitLab's monolith, and if so is it higher or lower than managing that?
1. Do we require Runway features that are not in the list of supported features?
1. In terms of tooling handover, does the team have any experience with Runway and are they willing to take it over if it is deployed using that method?

Which can be visualized as the following mermaid diagram:

```
mermaid
graph TD
    A[Is the tool stateless and does it benefit from being auto scaled?] -->|Yes| B[Does it require an infrastructure that does not fit into GitLab?]
    A -->|No| F[Not a good fit for Runway]
    B -->|Yes| C[Is there some complexity to managing the infrastructure of the tool that would justify it being outside of GitLab's monolith?]
    B -->|No| F
    C -->|Yes| D[Are we adding another layer of complexity by having that infrastructure outside GitLab's monolith, and if so is it higher or lower than managing that?]
    C -->|No| F
    D -->|Lower complexity| E[Do we require Runway features that are not in the list of supported features?]
    D -->|Higher complexity| F
    E -->|No| G[Potential fit for Runway]
    E -->|Yes| F
```

How to start with Runway?

If the Candidate Services for Runway Deployment issue is still around, comment on it and set up a chat with the team to discuss whether Runway would actually be a good fit for the intended purpose. Then the docs offer a good onboarding guide to get started.

A conversation should be started with the team that will own the tool down the road about using Runway as a tool as part of the tooling handover epic.

SECTION: security/product-security/security-platforms-architecture/product-security-engineering/runbooks/team-member-upskilling.md

Team Member Upskilling Runbook

This runbook is a collection of resources for new or existing Product Security Engineers, or for those looking to build a body of work to enhance their skills and knowledge in product security. This runbook is designed to provide guidance and resources for continuous learning and professional development.

1. Core Competencies

Team members should work to become proficient in the following core competencies:

- Web application security
- Secure coding practices
- Threat modeling
- Risk assessment and management

Team members should also have 'good' knowledge of the following areas; enough to know how to go and learn more should the need arise:

- Cloud / network security
- Cryptography

2. Learning Paths

These learning paths are designed to help new team members in Product Security Engineering get up to speed and make meaningful contributions. They provide a balance of quick wins and longer-term skill development, focusing on core competencies essential to our mission. Choose one or more paths that align with your interests and the team's current priorities.

1. "Paved Road & Defense-in-Depth" Learning Path

Essential:

-] Analyze a historic GitLab vulnerability (from release posts or our [internal or public Reproducible Vulnerabilities page) or identify a class of recurring issues from Bug Bounty

Stats or HackerOne

- [] Design a solution that makes it easier for developers to avoid this class of bug ("paved road"), and/or prevents or mitigates the class of bug (defense-in-depth)
- [] Document your solution design in an [issue (and/or epic & child items)]
 - Have the solution peer reviewed by a teammate
 - Iteratively refine it until it's in the ~workflow::ready for development state

Recommended:

- [] [Add it to a milestone and implement your solution]
- [] Reproduce a historic vulnerability on a local version of GitLab
- [] [Create a threat model that encompasses the vulnerability you chose]
- [] Compare your designed solution to our actual patches and defense-in-depth measures

2. "Secure GitLab with GitLab" Learning Path

Essential:

- [] Identify an existing issue off our issue board or select a ~ProdSecEngCandidate issue which will help one of the Product Security teams better secure GitLab (the org) with GitLab (the product)
- [] Document your solution design in an [issue (and/or epic & child items)]
 - Have the solution peer reviewed by a teammate
 - Iteratively refine it until it's in the ~workflow::ready for development state

Recommended:

- [] [Add it to a milestone and implement your solution]
- [] Conduct a post-implementation review and propose improvements
- [] Collaborate with the Application Security team to identify other high-impact automation opportunities

3. "Security Upskilling and Collaboration" Learning Path

Essential:

- [] Review the following and identify areas that you want to learn more about:
 - [] GitLab's [Secure Code Guidelines]
 - [] GitLab's [Security Risk Quarterly]
- [] Create an issue or epic to track your learning efforts. Include:
 - [] Type of training / URL(s)
 - [] Your goal, estimated effort, & milestone
 - [] Links to G&D issues if you need to apply for budget (certifications, online courses, etc)

Recommended:

- [] Update this page with resources or processes you found helpful!

SECTION: security/product-security/security-platforms-architecture/risk-register/_index.md

The Product Security Risk Register (PSRR) is driven by and follows the StORM process from our Security Risk Team.

Criteria

The PSRR will only record weaknesses, inadequate design choices, or functional logic that exposes GitLab, the product, to compromises and leaks. The goal is to track risks that have important and cascading consequences for GitLab, the product, as a whole.

Specific vulnerabilities or components based vulnerabilities (equivalent to what would be tracked as a single CVE) should not be part of the PSRR.

Objectives

- Serves as a centralized tool for identifying, evaluating, mitigating, and monitoring systemic risks that can impact GitLab's operations and reputation, but that doesn't meet the criteria to be tracked by the Security Risk team. Specific vulnerabilities or components based vulnerabilities should not be part of the PSRR.
- Identify clear DRIs to help in reducing and mitigating the risk. They will ensure that all stakeholders are aware of potential risks and are working collaboratively to manage them. The DRI is responsible for updating the risk register issues to reflect the progress made in reducing the risks.

Product Security teams are responsible for communicating the risk and coordinating mitigation with the risk's stakeholders, such as Engineering and Product. Due to their systemic impact, risks tracked in the PSRR are cross-functional and thus risk reduction will require cross-functional collaboration.

You can find this list in the StORM repository (internal), or consult the dashboard here (internal).

To create a new issue follow this link (internal). Make sure to reference the guide to defining well-articulated risks.

What risks should be in the PSRR?

- Only weaknesses, inadequate design choices, or functional logic that exposes GitLab-produced technologies to cyber risks.

Review

Risks tracked in our risk register are reviewed on a monthly basis by Product Security leadership.

SECTION: security/product-security/security-platforms-architecture/risk-register/well-

If you believe you have identified a systemic product or platform security risk in GitLab, the first step is to document the problem statement in the form of a Well-Articulated Risk. Only after a Well-Articulated Risk is documented can Product Security begin the process to triage, assess, prioritize, and treat it. Those downstream processes are out of scope for initial risk documentation.

What Makes a Well-Articulated Risk

A well-articulated product security risk clearly communicates:

- 1. Problem Statement: A specific, concise description of the security vulnerability, design weakness, functional logical issue, or capability gap that exposes GitLab (the product) to compromise.
- 1. Potential Impact: Clear explanation of the potential harm to GitLab and/or its customers if the risk remains unaddressed, including possible business impacts, data compromise scenarios, service or operational disruptions, etc.
- 1. Supporting Evidence: Concrete trends, data points, or proofs of concept that substantiate the risk is real and not merely theoretical that can help inform prioritization.
- 1. Scope and Scale: Information about how widespread the risk is and what parts of the product or infrastructure are affected.

If the risk you are describing is complex, consider including a flowchart or diagram to explain it effectively.

When you are initially reporting a risk, you may have ideas about possible solutions, or you may already know of challenges or blockers that need to be addressed. That's great! After you have created your Well-Articulated Risk, add those to the Issue as comments. Those will feed into the downstream risk treatment planning process.

What a Well-Articulated Risk is NOT

- 1. A Solution Statement: "We need to implement X technology" or "build Y feature" without explaining the underlying risk.
- 1. A Feature Request: Describing desired functionality without connecting it to an actual security exposure.
- 1. An Individual Vulnerability: Specific CVEs or component-based vulnerabilities belonging elsewhere.
- 1. A Vague or Generalized Problem: "Our security posture is weak" without specific details.

Additional Guidance

If you need help drafting an initial risk, see this Risk Drafting Guidance from our Security Risk Team. Alternatively, reach out to the Security Platforms and Architecture Team in Slack in security-spa.

SECTION: security/product-security/security-platforms-architecture/security-

architecture/_index.md

<link rel="stylesheet" type="text/css" href="/stylesheets/biztech.css" />

Team Information

- Team Pages
 - Handbook Page
 - GitLab Sub-Group
- Contact Info
 - Slack Channels
 - security-architecture: main channel to engage with the team
 - Tag on GitLab: @gitlab-com/gl-security/security-architecture

Overview

Security Architects are the trusted security advisors of GitLab Engineering. Security Architecture is a natural extension of the greater Architecture initiative at GitLab. It is the preliminary and necessary work to build software with security considerations.

Objectives

Security Architecture protects the organization from cyber harm, and support present and future business needs by:

- Preventing Security from being an afterthought
- Defining Security Architecture Principles
- Aligning with our security sub-departments requirements and expectations
- Assisting other departments in the design and architect of new features, services, products.
- Identifying the right SMEs and DRIs on the security side
- Driving security initiatives and features

The process is designed with these constraints in mind:

- aligned with our values
- asynchronous
- self-service as much as possible
- avoid being a bottleneck in the software development life cycle
- deliberately simple and concise
- automated as much as possible
- DRY with strong notes

Scope of Security Architecture

Any change in our product offering (whether it is a feature, a service, or an acquisition), that would impact our security posture. Our security posture is defined by:

- the components we build upon
- the components we embed
- everything infrastructure
- 3rd party services
- software architecture
- reference architectures

Security Architecture Requirements

Application Security

The Application Security team provides guidelines and requirements to follow during all the life cycle of source code:

- AppSec projects policies

InfraSec

<!-- Using this page until infrasec requirements are in the handbook -->

- Security Requirements for Development and Deployment

Compliance

- GitLab Audit Logging Policy

Cryptography

- Do not roll your own crypto (also one of our Security Architecture Principles)
- Reference our GitLab Cryptography Standard

<!-- Add FIPS and FedRamp requirements here when available -->

Security Architecture Principles

The Security Architecture Principles are not requirements nor decisions, but something in between.

Our principles are based on two simple pillars:

1. Least privilege
1. Network isolation

They are detailed below with the principles taken from the book Software Systems Architecture (see references) and this ACCU 2019 related video. These are very close to the OWASP Security Design Principles but are easier to understand and apply.

```
<style>
.security-architecture-principle {
  display: inline-block;
}
</style>
```

```
<details>
<summary>
```

Assign the least privilege possible{.security-architecture-principle}

```
</summary>
```

Why

Broad privileges allow malicious or accidental access to protected resources.

How

- Give only the minimum level of access rights (privileges) that is necessary to a user or service to complete an assigned operation. This right must be given only for a minimum amount of time that is necessary to complete the operation.
- Do not use administrative accounts for application access
- Use separate accounts for sensitive data

Examples

- Run service processes as their own users with exactly the set of privileges they require
- Grant read-only permissions when no updates are required
- When updates are required, limit to the scope to the target resource only

Links

- <<https://owasp.org/www-community/AccessControlprinciple-of-least-privilege>>
- <<https://csrc.nist.gov/glossary/term/leastprivilege>>
- <<https://handbook.gitlab.com/handbook/security/security-and-technology-policies/access-management-policy/>>

```
</details>
```

```
<details>
<summary>
```

Separate responsibilities{.security-architecture-principle}

```
</summary>
```

Why

Limit the blast radius of successful attacks: When one part of the system is compromised, the whole system is not.

Make attacks less attractive.

How

- Compartmentalize responsibilities and privileges
- Separation of duties: the successful completion of a single task is dependent upon two or more conditions
- Don't store secrets along with other non-sensitive data (like settings), even if secrets are filtered out

Examples

- A system/service that only needs to read git commits should not be able to access user data
- GitLab team members don't have access to billing data, nor anything else classified red data

Links

- OWASP Access Control Models
- <https://en.wikipedia.org/wiki/Compartmentalization%28informationsecurity%29>

</details>

<details>

<summary>

Trust cautiously{.security-architecture-principle}

</summary>

Why

- Many security problems caused by inserting malicious intermediaries in communication path

How

- Assume unknown entities are untrusted
- Have a clear process to establish trust
- Validate who or what is connecting
- Always use a kind of authentication (certificate, password, ...)
- Network controls
- Do not dynamically load 3rd party code

Examples

- Services can't be considered as secure as soon as they are not exposed to the Internet.

SSRF can let attackers freely access them.

- The best way to authenticate users is to apply this general security principle: Provide something you know (ex: password), and something you own (ex: certificate). This is what we apply with MFA, for example by providing a password you know, along with a TOTP that is generated by an application.
- Downloading 3rd party libraries or scripts at runtime can lead to many security issues, including cache poisoning, XSS, and whatnot. Without checking the integrity of the external asset, malicious actors can tamper the files, like this example of BGP Hijacking

Links

- Zero Trust at GitLab

</details>

<details>

<summary>

Simplest solution possible{.security-architecture-principle}

</summary>

Why

- Simple solutions are easier to deploy, maintain, and secure
- Aligned with our Iteration and Efficiency values
- Security requires understanding of the design
- Complexity increases exponentially
- Attack-ability or attack surface of the software is reduced

How

- Avoid complex failure modes, implicit behaviours, unnecessary features
- Use well-known, tested, and proven components
- Avoid over-engineering and strive for MVCs instead

Examples

- Introducing a new server in GitLab means updating Omnibus builds, Helm charts, our reference architectures, our docs, and so on. This is something to balance carefully against the benefits of adding a component which seem to be a perfect fit.

Links

- Keep it simple, stupid
- Complexity and exponential change

</details>

<details>

<summary>

Audit sensitive events{.security-architecture-principle}

</summary>

Why

- Provide record of activity
- Deter wrong doing
- Provide a log to construct that past
- Provide a monitoring point

How

- Record all security significant events in a tamper-resistant store
- Provide notifications for all sensitive events

Examples

- Enable GuardDuty in AWS or Cloud Audit Logs in GCP to record activity and detect malicious intent.
- Leverage a SIEM (Devo for gitlab.com) to collect, normalize, and analyze logs.
- Provide notifications to users when:
 - Changes to their accounts
 - New keys generated or added to their accounts
- Generate security events (could be Slack notifications) for unusual activity:
 - Signal passing a threshold (rate limiting in action)
 - Component signature not matching
 - Unauthorized access to sensitive resources

Links

</details>

<details>

<summary>

Fail securely & use secure defaults{.security-architecture-principle}

</summary>

Why

- Default passwords, ports and rules are "open doors"
- Failure and restart states often default to "insecure"

How

- Force changes to security sensitive parameters
- Think through failures - to be secure but recoverable
- Unless a subject is given explicit access to an object, it should be denied access to that object,
aka Fail Safe Defaults.

Examples

- Do not trust invalid/expired TLS certificates
- Some components like Grafana come with a default admin/admin user/password.
- Related to above, some components might fail over to a plain user/password authentication (with default credentials) under certain conditions, like a service not reachable.
- Some frameworks tend to render error pages with details that should not be shared, like hostnames and paths, when they cannot connect to some resources.

Links

- <https://owasp.org/www-community/Failsecurely>

</details>

<details>
<summary>

Never rely upon obscurity{.security-architecture-principle}

</summary>

Why

- Hiding things is difficult, someone is going to find them, accidentally or on purpose
- We're a very transparent company and are more likely to share implementation details, sometimes leaking something sensitive.
- Offboarded employees leave with sensitive knowledge. While tokens can be rotated, we can't ensure this knowledge won't leak

How

- Assume attacker with perfect knowledge

Examples

- Recon can help attackers find servers that are not publicly documented. These servers could expose vulnerable components, and lead to east-west movement.
- Changing the path to a admin section won't prevent attackers from finding it eventually.

Links

- <<https://securitytrails.com/blog/security-through-obscurity>>

</details>

<details>

<summary>

Implement defense in depth{.security-architecture-principle}

</summary>

Why

- Systems do get attacked, breaches do happen, mistakes are made
- Minimize blast radius: One component compromised should not compromise the whole system
- Prevent SSRF

How

- Don't rely on a single point/layer of security:
 - Secure every level
 - Stop failures at one level propagating
- Encrypt data at rest and in transit (according to GitLab's encryption policy)
- Use vulnerability scanners
- Close unnecessary ports and disable unused features

Examples

- A resource is well protected when accessed with the UI, but could be more exposed through the API.
- Accounts are locked when too many attempts, in order to avoid brute-force attacks.
- OS execution can lead to bypass all application security layers, because the execution occurs outside of the application.
- Unnecessary open ports and enabled features may lead to authentication bypass and other weaknesses. They increase the exposure of an application.

Links

- <[https://en.wikipedia.org/wiki/Defenseindepth\(computing\)](https://en.wikipedia.org/wiki/Defenseindepth(computing))>
- Zero Trust at GitLab

</details>

<details>

<summary>

Never invent security technology{.security-architecture-principle}

</summary>

Why

- Security technology is difficult to create, and avoiding vulnerabilities is difficult
- It takes years to secure and mature new security technologies
- They are expected to be perfect (sort of)

How

- [Do not roll your own crypto]
- Use well-known and proven components
- In doubt, always involve the right SMEs

Examples

- Do not implement SSO from scratch

Links

- Rainbow tables are used to cache the output of cryptographic hash functions, which can be used for cracking password hashes.
- Perils of the default bcrypt cost factor:
Even when using a famous crypto library, its configuration can lead to weaknesses.

</details>

<details>

<summary>

Find the weakest link{.security-architecture-principle}

</summary>

Why

- A system is just as secure as its weakest link
- Over time, new vulnerabilities are discovered, and a component might suddenly become the new weak link

How

- Threat model the system, repeat, iterate.
- Identify central components that

- share more privileges than the others
- have more connections to other components
- are entrypoints (login modules, APIs, ...)
- Run Dependency Scanning
- Avoid weak ciphers and algorithms
- Sometimes consider the humans (users) as the weakest link. Phishing is still widely used for a good reason

Examples

- Some resources are very well protected in the UI, and never exposed to unauthorized users. Yet, if the API is not correctly implementing security controls, these resources could be passed as raw models without filtering sensitive data.
- Data encrypted in transit but not at rest.
- The weakest link could also be a user. Not enforcing strong passwords and MFA could lead to sensitive data exposure, but users can also do harmful actions without being aware of it.
- OS (system) commands often leads to bypassing most, if not all, the security controls of an application. It is a common vector for RCEs and should be avoided as much as possible.

Links

- RCE when removing metadata with ExifTool
- Log4Shell: RCE 0-day exploit found in log4j 2, a popular Java logging package

</details>

Measuring results

TODO (discussed in <https://gitlab.com/gitlab-com/gl-security/product-security/security-architecture/general/-/issues/49>)

References

- Software Systems Architecture (ISBN-13: 978-0321718334)
- SABSA
- TOGAF
- O-ESA
- OSA
- NIST CSF
- CIS Critical Security Controls
- Cyber Defense Matrix
- AWS Well Architected Framework
- OWASP Developer Guide Reboot
- Google Cloud: Optimize your system design using Architecture Framework Principles

SECTION: security/product-security/security-platforms-architecture/security-architecture/zero-trust.md

Zero Trust

As part of raising that bar, GitLab is implementing Zero Trust, or the practice of shifting access control from the perimeter of the org to the individuals, the assets and the endpoints. You can learn more about this strategy from the Google BeyondCorp whitepaper: [A New Approach to Enterprise Security](#).

In our case, Zero Trust means that all devices trying to access an endpoint or asset within our GitLab environment will need to authenticate and be authorized. Because Zero Trust relies on dynamic, risk-based decisions, this also means that users must be authorized and validated: what department are they in, what role do they have, how sensitive is the data and the host that they are trying to access? We're at the beginning stages in our Zero Trust roadmap, but as we move along in the journey, we'll document our lessons learned, process and progress in our Security blog.

To learn more about the concept of Zero Trust and our roadmap for implementation, see this GitLab presentation from GoogleNext19: <https://www.youtube.com/watch?v=DrPiCBtaydM>

You can also check out our Zero Trust Networking (ZTN) blog series where we detail the ZTN implementation challenges we foresee ahead, some we've already managed to work through, and where we'll go from here:

- Part one: The evolution of Zero Trust
- Part two: Zero Trust at GitLab: Problems, goals, and coming challenges
- Part three: Zero Trust at GitLab: The data classification and infrastructure challenge
- Part four: Zero Trust at GitLab: Mitigating challenges with data zones and authentication scoring
- Part five: Zero Trust at GitLab: Implementation challenges
- Part six: Zero Trust at GitLab: Where do we go from here?

Head over to the [/r/netsec](#) subreddit to see our October 29, 2019 Reddit AMA on Zero Trust where we fielded questions around our ZTN implementation, roadmap, strategy and more.

Identity is a critical element of the implementation of a ZTN framework. GitLab is moving forward with an implementation of Okta to allow us to standardize authentication for Cloud Application access and implement user-friendly SSO. See our [Okta](#) page for more details.

Why We Don't Have a Corporate VPN

In many enterprise environments, virtual private networks (VPN) are used to allow access to less secured resources, typically also protected by an enterprise firewall. Adding corporate VPN connectivity only marginally improves the security of using those systems and assumes a network perimeter is in place. At GitLab, as an all remote company, we do most of our work using other Software-as-a-Service (SaaS) providers that we rely on to maintain confidentiality of communication and data.

In relation to Zero Trust, a corporate VPN is a perimeter, which ZTN architecture deemphasizes as a basis for making authorization decisions. Current access to critical systems is managed through alternative controls.

While a corporate VPN is not implemented at this time, there are other valid use cases for which individual team members may still wish to use a personal VPN, such as privacy or preventing traffic aggregation. Team members that wish to use a personal VPN service for any reason may still expense one.

For the use case of laptop usage in untrusted environments, such as coffee shops and coworking spaces, team members should prioritize a baseline of always-on host protections, such as up-to-date security patching, host firewalls, and antivirus, by following the Laptop Software Configuration Standards. That said, a personal VPN may provide additional protections in these situations. For more on personal VPNs see the Personal VPN page.

SECTION: security/product-security/security-platforms-architecture/security-interlock/_index.md

What We Do

The Security Division's dogfooding of GitLab security features isn't just a company practice; it's a strategic imperative. When we use our own security products internally, we identify usability challenges before our customers encounter them, and we deliver a more compelling go-to-market narrative that inspires trust and confidence in our solutions. Our firsthand experience informs product roadmaps and prioritization decisions, bridging the gap between theoretic functionality and practical implementation. GitLab's Security Division is a canary for enterprise security team needs. If we can't use our own security features effectively, our customers likely can't either.

Beyond simply testing existing features, Security, Engineering, and Product actively partner to co-create security capabilities directly within the GitLab platform. This co-creation approach ensures that real-world security functionality and risk mitigation controls · developed by security practitioners for security practitioners · become core elements of the product. By directly integrating our security expertise and requirements into GitLab, we deliver practical solutions that address enterprise security challenges.

While GitLab has always emphasized dogfooding and cross-functional collaboration, we're increasing our focus and formalizing these efforts in FY26.

Security Interlock Workstreams

In FY26, we are executing on our Security Interlock vision by launching three distinct, parallel workstreams.

Workstream Name	Workstream Description
-----------------	------------------------

-----	-----
-------	-------

Customer Zero	Security's collaboration throughout the development and launch of new
---------------	---

features, including requirements delivery, validation of intended functionality and mockups, and alpha/beta testing |

| Dogfooding Existing Features | Ensuring Security's ability to reliably use and derive value from existing features |

| Internal Co-Create Process | Direct contributions of security features, paved paths, and guardrails to the GitLab platform from the Security Division using the Internal Co-Create Process |

We're actively formalizing these workstreams now. We are intentionally starting small with 1-2 target features for each to refine our collaborative interfaces before scaling our efforts.

Key Drivers

The Security Interlock Initiative will have 3 designated leaders:

- The Security Platforms & Architecture team will represent on behalf of the Security Division.
- A Product DRI from the Sec Section.
- An Engineering DRI from the Sec Section.

While all Security Teams will continue to pilot applicable features and provide feedback, this leadership structure will create clear channels for communication and accountability. The goal is to streamline our collaborative efforts, ensure that consolidated, actionable feedback reaches product teams efficiently, and facilitating coordinated development of security features that address real-world needs. By centralizing these inte